

MESATECH CLOUD

Informe técnico · Evaluación parcial 2

Mensajería con RabbitMQ y persistencia independiente en AWS

DSY1107 · Desarrollo Cloud Native I

Ari Araya · Giannina Guerrero · Nicolás Barra · Skarlett Tropan

Noviembre de 2026

Índice

1. Introducción **3**
2. Objetivos **3**
2.1 Objetivo general 3
2.2 Objetivos específicos 3
3. Cambios respecto de la EP1 ..... **4**
4. Arquitectura de despliegue **4**
4.1 Flujos implementados 5
4.2 Componentes y propiedad de los datos 5
5. Infraestructura en AWS **7**
5.1 Instancias EC2 7
5.2 Instancias RDS 8
5.3 Versiones del entorno 9
5.4 Configuración de conexión sin contraseñas 10
6. Instalación de RabbitMQ desde cero **10**
6.1 Instalación de Docker Engine y Docker Compose 10
6.2 Archivo compose.yaml 11
6.3 Validación, arranque y diagnóstico 12
6.4 Interfaz de administración 13
7. Red y seguridad **13**
7.1 Security groups 13
7.2 Credenciales y firewall del sistema operativo 14
8. Mensajería con RabbitMQ **15**
8.1 Recursos del virtual host 15
8.2 Permisos de los usuarios de aplicación 16
8.3 Contrato del evento categoria.creada 17
8.4 Confirmaciones de publicador y consumidor 18
8.5 Manejo de fallas 19
9. Implementación del consumidor en Solicitudes **19**
9.1 Secuencia de procesamiento 20
9.2 Uso de la copia local 20
9.3 Verificación en entorno local 20
9.4 Estado del publicador en Catálogo 21
10. Continuidad con la EP1 ..... **21**
11. Plan y resultados de pruebas P1 a P8 ..... **21**
11.1 Consulta de verificación de la copia local 23
11.2 Reflexión sobre publicación, confirmaciones, persistencia y fallos 23
12. Limitaciones y trabajo pendiente **24**
13. Conclusiones **24**
14. Responsabilidades del equipo **25**
Referencias **25**

1. Introducción

El presente informe documenta la segunda evaluación parcial del proyecto MesaTech Cloud, desarrollado por el equipo en la asignatura Desarrollo Cloud Native I de Duoc UC. La plataforma conserva el caso de soporte tecnológico para PYMEs presentado en la EP1 y amplía su arquitectura para que los servicios de Catálogo y Solicitudes operen con infraestructura y persistencia independientes en AWS.

En esta etapa, Solicitudes necesita disponer de las categorías de atención que mantiene Catálogo sin consultar la base de datos de otro servicio. Para resolverlo, el equipo incorporó un broker RabbitMQ instalado desde cero con Docker Compose en una instancia EC2 propia. Catálogo guarda cada categoría nueva en su instancia RDS y publica un evento. Solicitudes consume ese evento y conserva una copia local en su propia instancia RDS.

El documento describe los cambios respecto de la EP1, la arquitectura de despliegue, el inventario de EC2 y RDS, la configuración de red, la instalación del broker y el diseño de la mensajería. También presenta el plan de pruebas P1 a P8 y una reflexión sobre publicación, confirmaciones, persistencia y fallos.

Las evidencias que aún no están disponibles se señalan en recuadros de color ámbar, con su responsable. Los resultados de las pruebas que todavía no se ejecutan se mantienen como pendientes y no se completan con valores supuestos.

2. Objetivos

2.1 Objetivo general

Evolucionar MesaTech Cloud hacia una arquitectura con dos microservicios Spring Boot desplegados en instancias EC2 diferentes, cada uno con su propia instancia Amazon RDS, integrados mediante mensajería asíncrona con RabbitMQ y sin perder la seguridad de acceso construida en la EP1.

2.2 Objetivos específicos

- Separar Catálogo y Solicitudes en instancias EC2 distintas y asignar a cada servicio una instancia RDS exclusiva.
- Instalar RabbitMQ desde cero en una tercera EC2 con Ubuntu 24.04 LTS, Docker Engine y el plugin Docker Compose.
- Configurar el virtual host, los usuarios de aplicación, el exchange, la cola y el binding del dominio MesaTech.
- Definir el contrato del evento `categoria.creada` y conservar el identificador de origen en la copia local.
- Restringir los puertos de AMQP, administración, bases de datos y SSH a orígenes justificados.

- Verificar la separación, la mensajería, la persistencia y la continuidad de la EP1 mediante las pruebas P1 a P8.

3. Cambios respecto de la EP1

La EP1 dejó operativa la plataforma con React y MSAL, Microsoft Entra ID, AWS API Gateway, un BFF con validación JWT y dos microservicios. En esa versión, las dos bases PostgreSQL se ejecutaban en un mismo contenedor Docker dentro de una EC2 de datos, y Solicitudes obtenía las categorías consultando la API de Catálogo.

La EP2 conserva el caso de negocio, los actores y las responsabilidades de cada servicio. Los cambios se concentran en la infraestructura y en la forma en que Solicitudes recibe las categorías. La Tabla 3.1 resume las diferencias.

Tabla 3.1. *Comparación entre la arquitectura de la EP1 y la de la EP2.*

Aspecto	EP1	EP2
Bases de datos	PostgreSQL 16 en Docker, con dos bases en un mismo contenedor	Dos instancias Amazon RDS PostgreSQL exclusivas y sin acceso público
Microservicios	Catálogo y Solicitudes en una misma EC2	Catálogo en la EC2 A y Solicitudes en la EC2 B
Categorías en Solicitudes	Consulta a la API de Catálogo	Copia local alimentada por el evento `categoria.creada`
Integración entre servicios	Llamadas HTTP coordinadas por el BFF	Mensajería asíncrona con RabbitMQ en la EC2 C
Broker de mensajería	No existía	RabbitMQ 4.2 con interfaz de administración, instalado con Docker Compose
Acceso seguro	Entra ID, API Gateway y BFF con JWT	Se mantiene sin cambios funcionales

El retraso propio del procesamiento asíncrono es aceptable para el caso. El formulario de creación de solicitudes muestra solo las categorías que ya llegaron a la copia local y permite actualizar el listado. De este modo, el usuario reconoce cuándo una categoría nueva está disponible, sin que la respuesta de Catálogo espere al consumidor.

4. Arquitectura de despliegue

La Figura 4.1, ubicada en una página horizontal al final de esta sección, presenta el diagrama real de despliegue en la región us-east-1 del Learner Lab de AWS Academy. Incluye React con MSAL, Microsoft Entra ID, API Gateway, el BFF, las tres instancias EC2 y las dos instancias RDS. Las flechas distinguen el acceso HTTP y SQL, la autenticación con Entra ID y el flujo de mensajes AMQP.

Catálogo, en la EC2 A, es el componente que publica. Solicitudes, en la EC2 B, es el componente que consume. Cada microservicio es propietario exclusivo de su base de datos.

El BFF no accede a ninguna RDS y no publica ni consume mensajes. Las direcciones IP se omitieron del diagrama por seguridad.

4.1 Flujos implementados

La arquitectura sostiene tres flujos. Los dos primeros corresponden a la creación y propagación de una categoría. El tercero corresponde al registro de solicitudes con las categorías ya recibidas.

Tabla 4.1. *Flujos del sistema y su estado al cierre del informe.*

Flujo	Recorrido	Estado
Creación de categoría	React → API Gateway → BFF → Catálogo → RDS Catálogo	Funcional desde la EP1; despliegue en la EC2 A pendiente
Propagación del evento	Catálogo → `catalogo.exchange` → `solicitudes.catalogo.queue` → consumidor → RDS Solicitudes	Consumidor implementado; publicador pendiente
Registro de solicitudes	React → API Gateway → BFF → Solicitudes → categorías locales y RDS Solicitudes	Implementado en código; despliegue en la EC2 B pendiente

4.2 Componentes y propiedad de los datos

La Tabla 4.2 indica la ubicación de cada componente, su responsabilidad y la base de datos que le pertenece.

Tabla 4.2. *Distribución de componentes y propiedad de los datos.*

Componente	Ubicación	Responsabilidad	Datos que posee
Catálogo y publicador	EC2 A	Mantener categorías y prioridades, guardar categorías y publicar eventos	RDS Catálogo
Solicitudes y consumidor	EC2 B	Mantener solicitudes y estados, consumir eventos y guardar la copia local	RDS Solicitudes
RabbitMQ	EC2 C	Recibir, enrutar y retener mensajes del dominio	Volumen del broker, sin datos de negocio
BFF	EC2 A o EC2 B	Seguridad y coordinación definidas en la EP1	Sin base de datos

El intercambio entre Catálogo y Solicitudes ocurre solo mediante eventos. El equipo no utiliza lecturas SQL cruzadas, archivos compartidos ni llamadas HTTP entre publicador y consumidor.

> *[Pendiente: definir y documentar la EC2 que aloja el BFF (A o B) y su puerto — responsable Nicolás]*

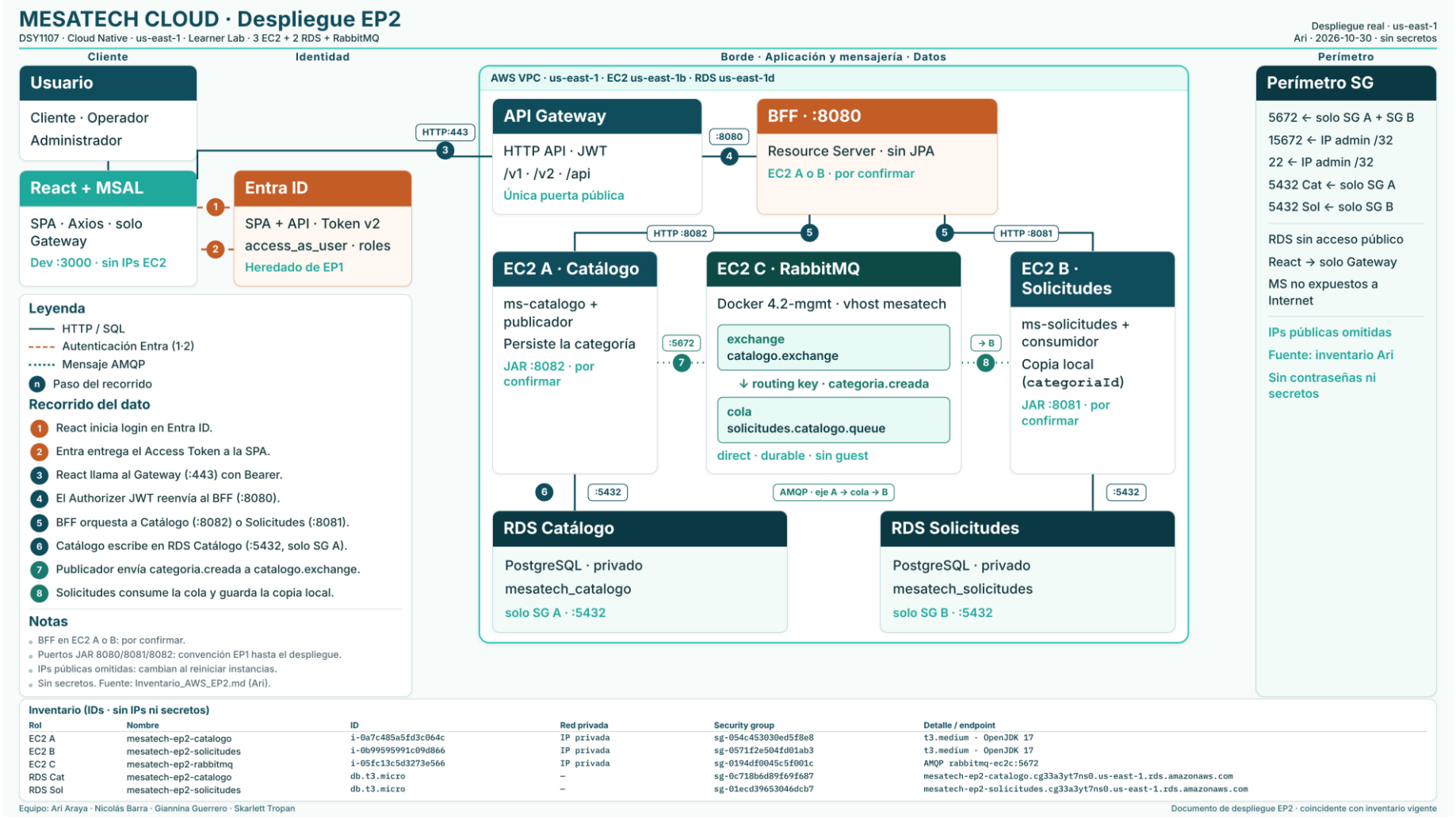


Figura 4.1. *Diagrama de despliegue de MesaTech Cloud en la EP2, con publicador, consumidor y propietario de cada base de datos.*

5. Infraestructura en AWS

Los recursos se crearon en el Learner Lab de AWS Academy, región us-east-1, entre el 29 y el 30 de octubre de 2026. Las tres instancias EC2 utilizan Ubuntu 24.04 LTS y el tipo t3.medium. Las dos instancias RDS utilizan PostgreSQL con clase db.t3.micro y fueron creadas sin acceso público.

5.1 Instancias EC2

La Tabla 5.1 identifica cada EC2 por su nombre, su instance ID y el proceso que aloja. Cada instancia tiene su propio security group.

Tabla 5.1. *Inventario de instancias EC2.*

Rol	Nombre	Instance ID	Tipo y zona	Proceso desplegado
EC2 A · Catálogo	mesatech-ep2-catalogo	`i-0a7c485a5fd3c064c`	t3.medium us-east-1b	ms-catalogo con publicador (despliegue pendiente)
EC2 B · Solicitudes	mesatech-ep2-solicitudes	`i-0b99595991c09d866`	t3.medium us-east-1b	ms-solicitudes con consumidor (despliegue pendiente)
EC2 C · RabbitMQ	mesatech-ep2-rabbitmq	`i-05fc13c5d3273e566`	t3.medium us-east-1b	Contenedor RabbitMQ con Docker Compose

Las direcciones IP privadas y públicas quedan registradas en el inventario interno del equipo y no se reproducen en este informe. Las IP públicas del laboratorio cambian al reiniciar las instancias, por lo que la comunicación entre servicios utiliza la red privada de la VPC.

La Figura 5.1 muestra la selección de la imagen Ubuntu Server 24.04 LTS al crear la EC2 del broker. La Figura 5.2 muestra las tres instancias en la consola, cada una con un instance ID distinto.

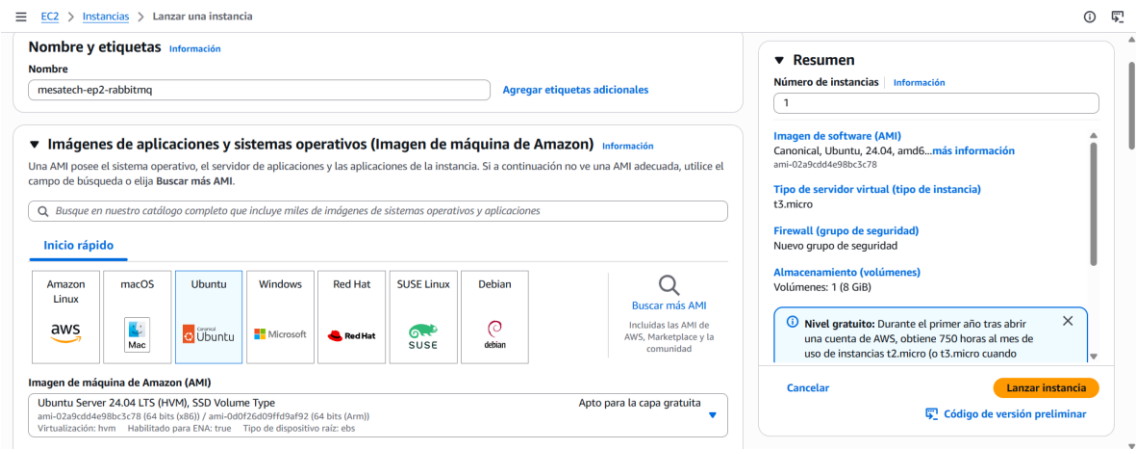


Figura 5.1. *Lanzamiento de la EC2 C con la AMI Ubuntu Server 24.04 LTS.*

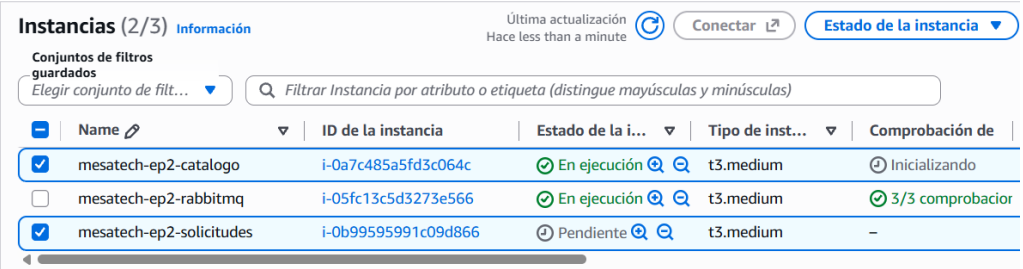


Figura 5.2. *Instancias EC2 del proyecto con sus identificadores.*

5.2 Instancias RDS

El proyecto utiliza dos instancias RDS independientes, no dos esquemas dentro de una misma instancia. La Tabla 5.2 presenta sus identificadores y endpoints.

Tabla 5.2. *Inventario de instancias RDS.*

Rol	Identificador	Endpoint (puerto 5432)	Base de datos	Acceso público
RDS Catálogo	mesatech-ep2-catalogo	`mesatech-ep2-catalogo.cg33a3yt7ns0.us-east-1.rds.amazonaws.com`	mesatech_catalogo	No
RDS Solicitudes	mesatech-ep2-solicitudes	`mesatech-ep2-solicitudes.cg33a3yt7ns0.us-east-1.rds.amazonaws.com`	mesatech_solicitudes	No

Ambas instancias usan el motor PostgreSQL, el puerto 5432, el grupo de parámetros `default.postgres18` y el driver JDBC `org.postgresql`. El equipo eligió PostgreSQL por continuidad con la EP1, cuyas bases ya utilizaban ese motor. La contraseña del usuario maestro de cada instancia se comparte solo por un canal privado del equipo.

En Solicitudes, las tablas se crean al iniciar la aplicación mediante JPA con la opción `ddl-auto=update`, lo que incluye la tabla `categorias_locales`. Cada security group de RDS acepta conexiones solo desde la EC2 de su microservicio, como se detalla en la sección 7.

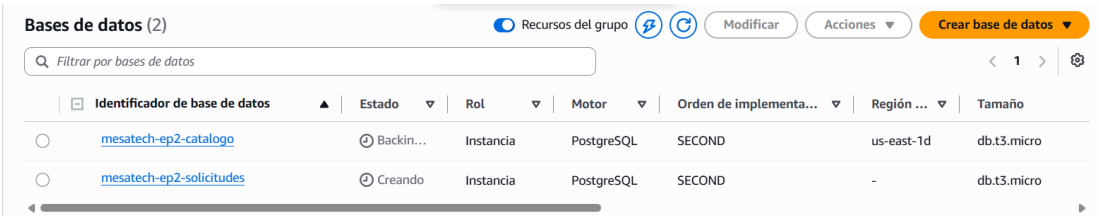


Figura 5.3. *Instancias RDS mesatech-ep2-catalogo y mesatech-ep2-solicitudes con motor PostgreSQL.*

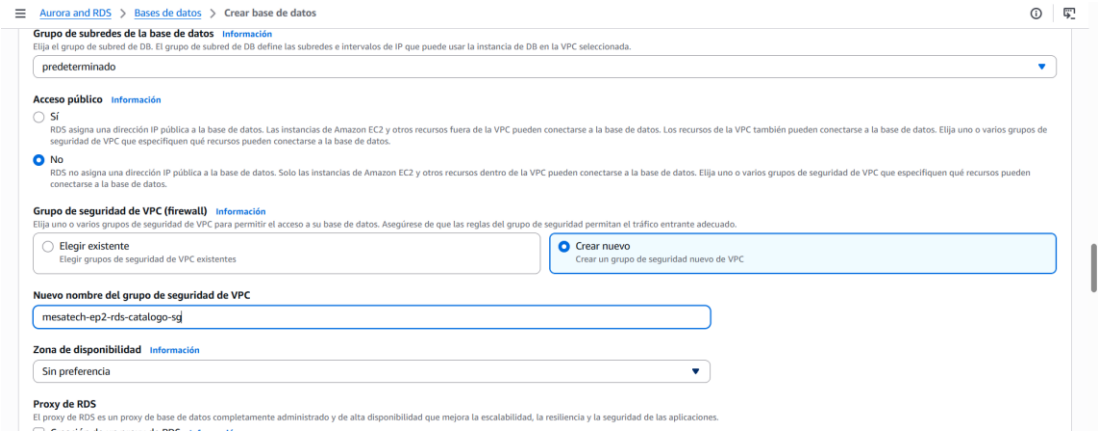


Figura 5.4. *Conectividad de RDS Catálogo sin acceso público y con security group dedicado.*

> *[Pendiente: consulta a las tablas de RDS Catálogo realizada desde la EC2 A — responsable Nicolás]*

> *[Pendiente: consulta a la tabla categorias_locales de RDS Solicitudes realizada desde la EC2 B — responsable Skarlett]*

> *[Pendiente: usuarios de aplicación en cada RDS, distintos del usuario maestro — responsable Ari]*

5.3 Versiones del entorno

La Tabla 5.3 registra las versiones utilizadas y la fuente que permite verificarlas.

Tabla 5.3. *Versiones de sistema operativo, plataforma y broker.*

Componente	Versión	Fuente de verificación
Sistema operativo de las EC2	Ubuntu 24.04 LTS	AMI seleccionada (Figura 5.1)
Java en las EC2 A y B	OpenJDK 17.0.20.1	Comando java -version (Figura 5.5)
Spring Boot	4.1.1, con Java 17 como versión objetivo	Archivo de proyecto de ms-solicitudes
Cliente AMQP	spring-boot-starter-amqp, versión gestionada por Spring Boot	Archivo de proyecto de ms-solicitudes
Docker Engine	29.9.2	Comando docker --version (Figura 6.1)
Docker Compose	v5.6.1	Comando docker compose version (Figura 6.1)
RabbitMQ	Imagen rabbitmq:4.2-management, RabbitMQ 4.2.9 y Erlang 27.3.4.19	Interfaz de administración (Figura 6.4)
PostgreSQL en RDS	Familia de parámetros default.postgres18	Consola de Amazon RDS

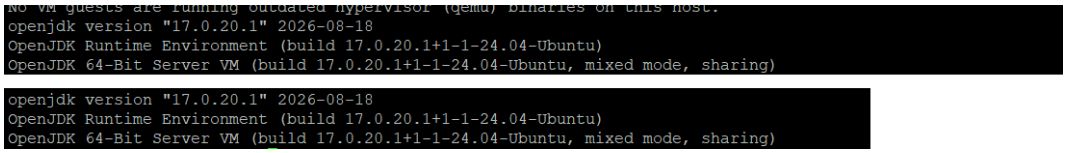


Figura 5.5. *OpenJDK 17 instalado en la EC2 A (arriba) y en la EC2 B (abajo).*

5.4 Configuración de conexión sin contraseñas

Ningún valor real de credenciales se guarda en el repositorio. Cada EC2 carga sus variables desde un archivo de entorno ubicado fuera del repositorio, y el repositorio contiene solo una plantilla con valores ficticios. La Tabla 5.4 muestra la configuración prevista para Solicitudes en la EC2 B.

Tabla 5.4. *Variables de entorno de ms-solicitudes en la EC2 B, sin secretos.*

Variable	Valor en la EC2 B	Propósito
`SOLICITUDES_DATASOURCE_URL`	jdbc:postgresql://mesatech-ep2-solicitudes.cg33a3yt7ns0.us-east-1.rds.amazonaws.com:5432/mesatech_solicitudes	Conexión exclusiva a RDS Solicitudes
`POSTGRES_USER` y `POSTGRES_PASSWORD`	Definidos fuera del repositorio	Credenciales de la base
`RABBITMQ_HOST` y `RABBITMQ_PORT`	IP privada de la EC2 C y puerto 5672	AMQP por red privada
`RABBITMQ_VHOST`	mesatech	Virtual host del proyecto
`RABBITMQ_USER` y `RABBITMQ_PASSWORD`	solicitudes_consumidor y contraseña fuera del repositorio	Usuario de aplicación del consumidor
`RABBITMQ_DECLARE`	false	El consumidor no declara recursos, porque el broker ya los contiene
`ENTRA_ISSUER_URI` y `ENTRA_AUDIENCE`	Valores del tenant de la EP1	Validación del JWT

La configuración de Catálogo sigue el mismo patrón, con el endpoint de RDS Catálogo y el usuario `catalogo_publicador`. Su detalle se incorporará junto con el despliegue del publicador.

6. Instalación de RabbitMQ desde cero

La EC2 C se creó sin broker preconfigurado. Sobre Ubuntu 24.04 LTS, el equipo instaló Docker Engine y el plugin Docker Compose desde el repositorio oficial de Docker, construyó el archivo `compose.yaml` y levantó el contenedor. No se utilizó Amazon MQ ni una instalación directa de RabbitMQ en el sistema operativo.

6.1 Instalación de Docker Engine y Docker Compose

La instalación siguió la guía oficial de Docker para Ubuntu. Se instalaron los paquetes docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin y docker-compose-plugin, y el usuario

ubuntu se agregó al grupo docker. La Tabla 6.1 registra los comandos ejecutados y su propósito.

Tabla 6.1. *Comandos ejecutados en la EC2 C.*

Comando	Propósito
`sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`	Instalar Docker Engine y el plugin Compose
`sudo usermod -aG docker ubuntu`	Permitir el uso de Docker sin sudo en nuevas sesiones
`docker --version y docker compose version`	Verificar las versiones instaladas
`mkdir -p ~/rabbitmq && cd ~/rabbitmq`	Crear el directorio del despliegue
`nano .env`	Definir las credenciales del administrador fuera del repositorio
`nano compose.yaml`	Declarar el servicio RabbitMQ
`docker compose config --quiet`	Validar la sintaxis y la interpolación del archivo
`docker compose pull`	Descargar la imagen declarada
`docker compose up -d`	Crear y arrancar el contenedor en segundo plano
`docker compose ps`	Verificar el estado y los puertos publicados
`docker compose logs --tail=30`	Revisar el arranque del broker

La Figura 6.1 muestra el comando de instalación y las versiones resultantes de Docker Engine y Docker Compose.

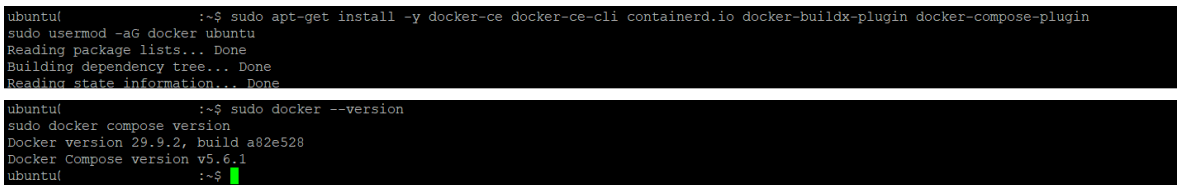


Figura 6.1. *Instalación de Docker en la EC2 C y versiones instaladas. El nombre de host de la terminal se ocultó.*

6.2 Archivo compose.yaml

El archivo `compose.yaml` declara un único servicio. Las credenciales del usuario administrador se leen desde un archivo `.env` local de la EC2 C, que no se versiona. La Figura 6.2 muestra el contenido del archivo, que no incluye secretos.

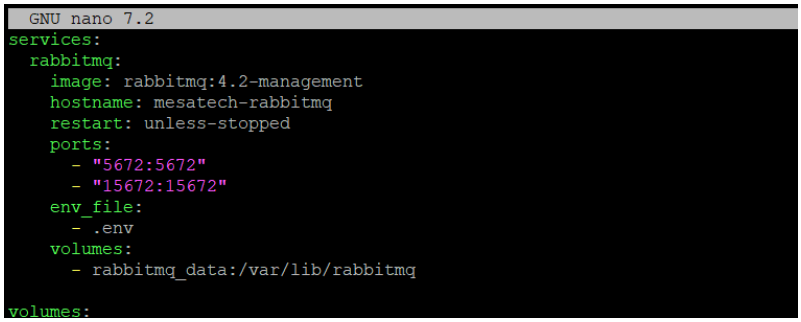


Figura 6.2. *Archivo compose.yaml del broker en la EC2 C.*

La Tabla 6.2 explica el propósito de cada configuración relevante del archivo.

Tabla 6.2. *Configuraciones de compose.yaml.*

Clave	Valor	Propósito
`image`	`rabbitmq:4.2-management`	Imagen oficial con etiqueta explícita que incluye la interfaz de administración
`hostname`	`mesatech-rabbitmq`	Nombre de nodo estable. RabbitMQ asocia su directorio de datos al nombre del nodo, por lo que conservarlo permite recuperar definiciones y mensajes al recrear el contenedor
`restart`	`unless-stopped`	Reinicia el contenedor ante fallos o reinicio de la EC2, salvo detención manual
`ports`	`5672:5672`	Publica AMQP para Catálogo y Solicitudes
`ports`	`15672:15672`	Publica la interfaz de administración HTTP
`env_file`	`.env`	Mantiene las credenciales de administración fuera del repositorio
`volumes`	`rabbitmq_data:/var/lib/rabbitmq`	Volumen con nombre que conserva definiciones y mensajes persistentes

> *[Pendiente: versionar compose.yaml y un .env.example con valores ficticios en el repositorio — responsable Ari]*

6.3 Validación, arranque y diagnóstico

La validación con `docker compose config --quiet` terminó sin mensajes de error. Luego, `docker compose pull` descargó la imagen y `docker compose up -d` creó la red, el volumen `rabbitmq_rabbitmq_data` y el contenedor `rabbitmq-rabbitmq-1`.

La salida de `docker compose ps` confirma el contenedor en estado Up y el mapeo de los puertos 5672 y 15672 del host hacia el contenedor (Figura 6.3).

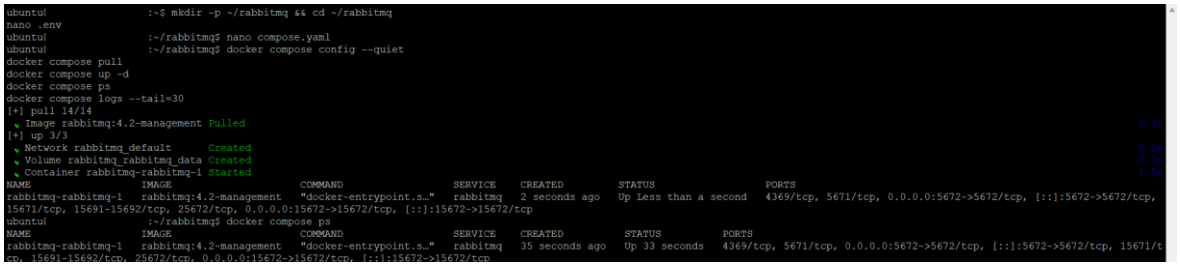


Figura 6.3. *Validación, descarga, arranque y estado del contenedor RabbitMQ. El nombre de host de la terminal se ocultó.*

El mapeo de puertos del contenedor y las reglas del security group son controles distintos y se verifican en conjunto. Docker publica los puertos en todas las interfaces de la EC2, por lo que el security group es el control que limita quién puede alcanzarlos.

> *[Pendiente: captura de docker compose logs y de rabbitmq-diagnostics status, sin secretos — responsable Ari]*

6.4 Interfaz de administración

La Figura 6.4 muestra la interfaz de administración con RabbitMQ 4.2.9, Erlang 27.3.4.19 y el nodo `rabbit@mesatech-rabbitmq` en ejecución. El nombre del nodo coincide con el hostname declarado en `compose.yaml`. La interfaz se accede solo desde la IP administrativa autorizada.

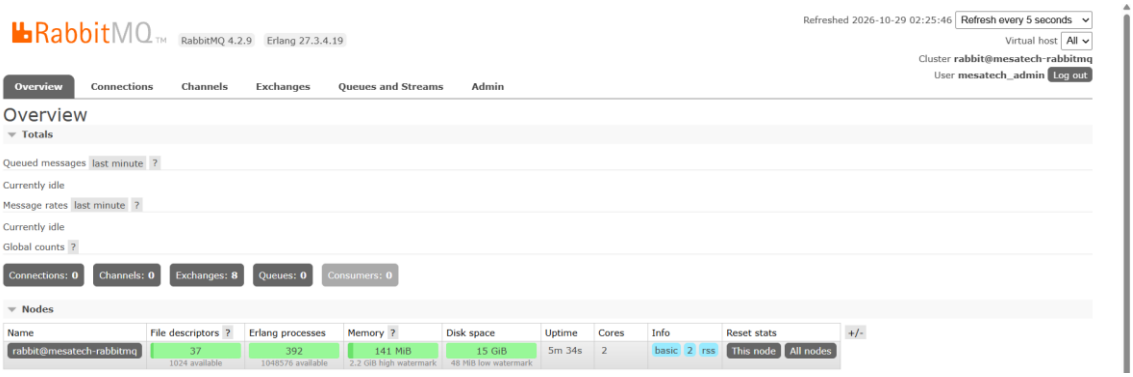


Figura 6.4. *Vista general de RabbitMQ en la EC2 C.*

7. Red y seguridad

La configuración de red sigue el principio de mínimo privilegio. Cada puerto se abre solo al origen que lo necesita y ninguna regla permite acceso mundial a RDS, AMQP o administración. La Tabla 7.1 presenta la matriz de comunicaciones.

Tabla 7.1. *Matriz de comunicaciones del proyecto.*

Destino	Puerto TCP	Origen autorizado	Propósito	Estado
EC2 C RabbitMQ	5672	Security groups de la EC2 A y de la EC2 B	AMQP por red privada	Configurado
EC2 C RabbitMQ	15672	IP administrativa con máscara /32	Interfaz de administración	Configurado
EC2 A, B y C	22	IP administrativa con máscara /32	Administración por SSH	Configurado
RDS Catálogo	5432	Security group de la EC2 A	Conexión de ms-catalogo	Configurado
RDS Solicitudes	5432	Security group de la EC2 B	Conexión de ms-solicitudes	Configurado
EC2 B Solicitudes	8081	Security group de la EC2 del BFF	Llamadas del BFF a ms-solicitudes	Pendiente
EC2 A Catálogo	8082	Security group de la EC2 del BFF	Llamadas del BFF a ms-catalogo	Pendiente
EC2 del BFF	8080	Integración de API Gateway de la EP1	Entrada de las APIs de negocio	Pendiente

7.1 Security groups

La Figura 7.1 muestra las reglas de entrada del security group de la EC2 C. Los puertos 15672 y 22 aceptan solo la IP administrativa con máscara /32, cuyo valor se ocultó en la captura. El

puerto 5672 acepta como origen los security groups de Catálogo y Solicitudes, en lugar de rangos de direcciones.

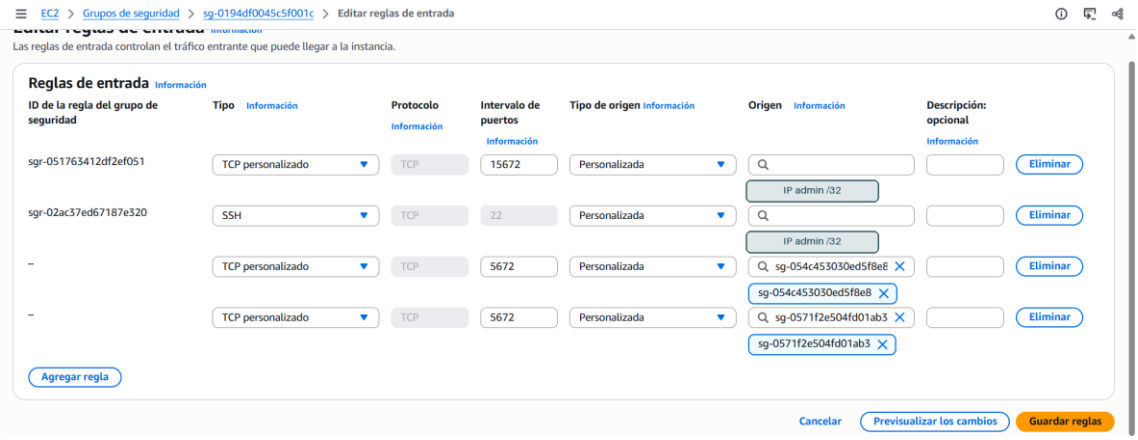


Figura 7.1. *Reglas de entrada del security group de la EC2 C.*

La Figura 7.2 muestra las reglas de las dos instancias RDS. Cada una acepta el puerto 5432 solo desde el security group de la EC2 de su microservicio propietario. Así, Solicitudes no puede conectarse a RDS Catálogo aunque conozca su endpoint.

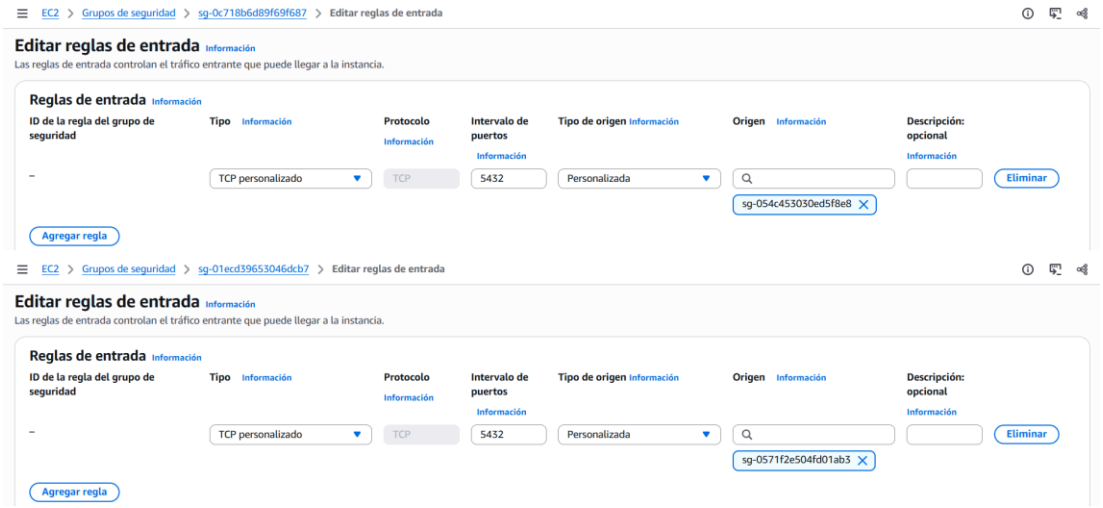


Figura 7.2. *Reglas de entrada de RDS Catálogo (arriba) y de RDS Solicitudes (abajo).*

7.2 Credenciales y firewall del sistema operativo

Las aplicaciones no utilizan el usuario guest de RabbitMQ ni el usuario administrador del broker. Las contraseñas de RabbitMQ y de RDS se mantienen fuera del repositorio y se distribuyen solo por un canal privado del equipo. Ninguna captura del informe muestra contraseñas.

> *[Pendiente: verificar y documentar el estado del firewall del sistema operativo (ufw) en las EC2 A, B y C — responsable Ari]*

> *[Pendiente: regla del security group de la EC2 B para el BFF hacia el puerto 8081 y acceso de despliegue a la instancia — responsable Ari]*

8. Mensajería con RabbitMQ

8.1 Recursos del virtual host

Los recursos se crearon desde la interfaz de administración con nombres del dominio MesaTech. La Tabla 8.1 los resume y las Figuras 8.1 a 8.5 muestran su estado en el broker.

Tabla 8.1. *Recursos de mensajería del proyecto.*

Recurso	Nombre	Configuración
Virtual host	`mesatech`	Aísla los recursos del proyecto
Exchange	`catalogo.exchange`	Tipo direct y durable
Cola	`solicitudes.catalogo.queue`	Durable, tipo classic y sin autoeliminación
Binding	`catalogo.exchange` → `solicitudes.catalogo.queue`	Routing key categoria.creada
Usuario publicador	`catalogo_publicador`	Sin etiquetas administrativas
Usuario consumidor	`solicitudes_consumidor`	Sin etiquetas administrativas
Usuario de administración	`mesatech_admin`	Etiqueta administrator, uso exclusivo del equipo

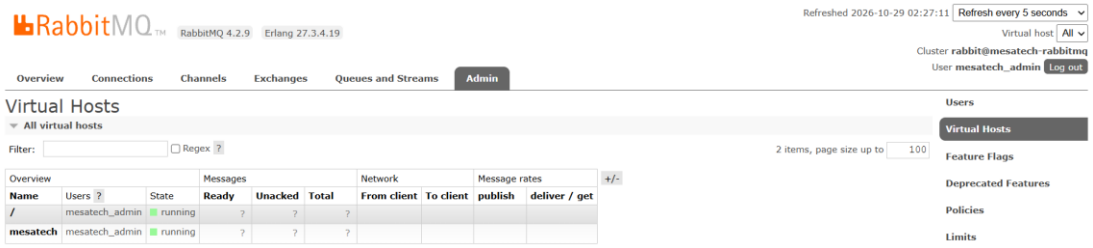


Figura 8.1. *Virtual host mesatech creado en el broker.*

En la Figura 8.2, los usuarios de aplicación figuran sin etiquetas y con acceso al virtual host mesatech. Solo `mesatech_admin` tiene la etiqueta administrator, y el usuario guest no aparece en el listado.

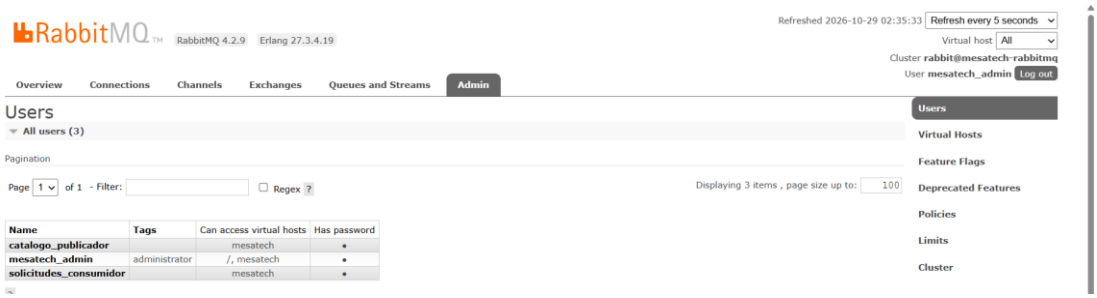


Figura 8.2. *Usuarios del broker y virtual host al que acceden.*

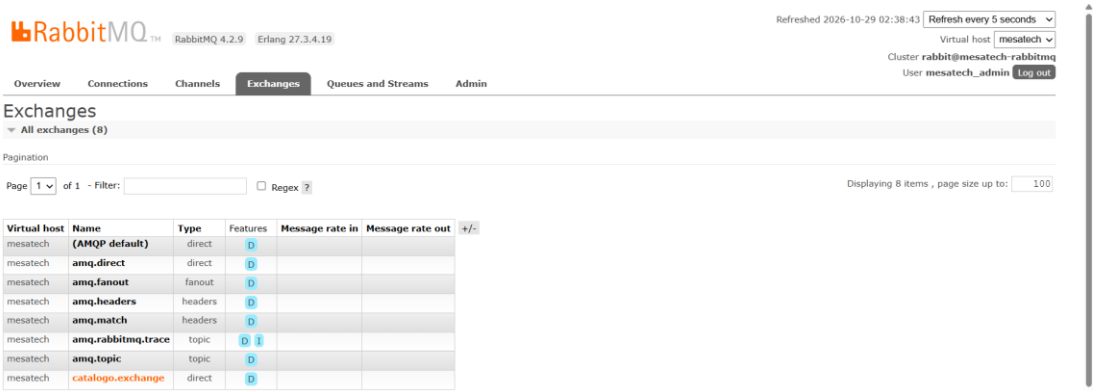


Figura 8.3. *Exchange catalogo.exchange de tipo direct y durable en el virtual host mesatech.*

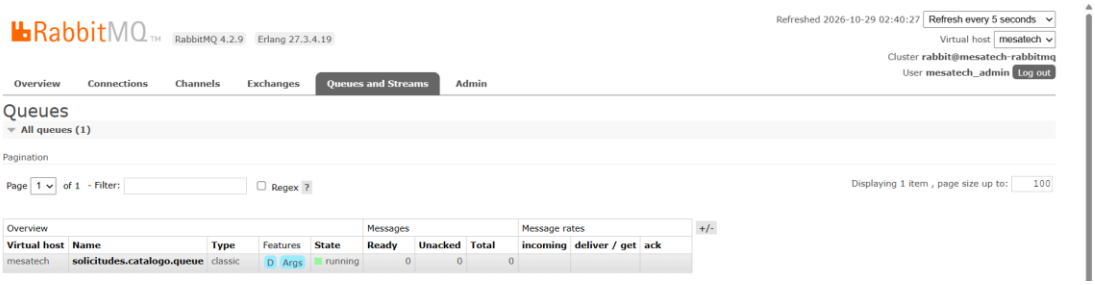


Figura 8.4. *Cola solicitudes.catalogo.queue durable y de tipo classic.*

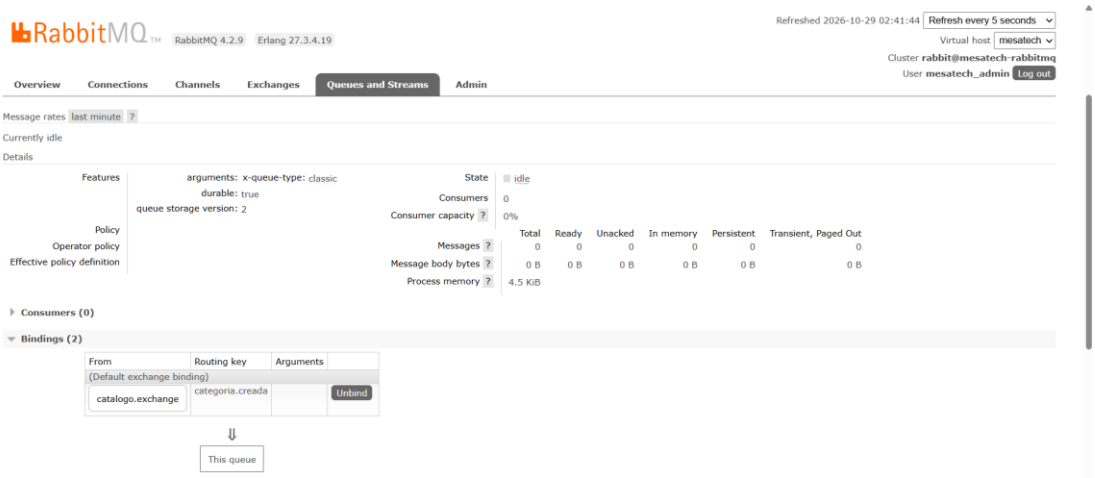


Figura 8.5. *Binding de la cola con catalogo.exchange mediante la routing key categoria.creada.*

8.2 Permisos de los usuarios de aplicación

RabbitMQ asigna permisos por virtual host en tres categorías. El permiso configure permite declarar o eliminar recursos, write permite publicar en un exchange y read permite consumir

desde una cola. El equipo entrega a cada usuario solo lo que su rol requiere, como resume la Tabla 8.2.

Tabla 8.2. *Asignación de permisos propuesta sobre el virtual host mesatech.*

Usuario	configure	write	read	Justificación
`catalogo_publicador`	Sin permisos	catalogo.exchange	Sin permisos	Solo publica eventos de categorías
`solicitudes_consumidor`	Sin permisos	Sin permisos	solicitudes.catalogo.queue	Solo consume la cola; con RABBITMQ_DECLARE=false no declara recursos

Los usuarios de aplicación no reciben etiquetas administrativas. La configuración efectiva de estos permisos se acreditará con la captura de la sección de permisos de cada usuario.

> *[Pendiente: captura de los permisos configure, write y read de catalogo_publicador y solicitudes_consumidor — responsable Ari]*

8.3 Contrato del evento categoria.creada

Al crear una categoría válida, Catálogo guarda el registro en RDS Catálogo y publica un mensaje JSON persistente en `catalogo.exchange` con la routing key `categoria.creada`. El contrato, en su versión 1.0, define cinco campos obligatorios (Tabla 8.3).

Tabla 8.3. *Campos del evento categoria.creada.*

Campo	Tipo	Formato	Regla
`eventId`	String	Identificador único, por ejemplo un UUID	Obligatorio y no vacío
`tipoEvento`	String	Literal categoria.creada	Debe coincidir con el tipo esperado
`fechaEvento`	Instant	ISO-8601 en UTC, por ejemplo 2026-10-29T15:00:00Z	Obligatorio
`categoriaId`	Long	Entero positivo	Identificador original de Catálogo, usado como clave primaria de la copia
`nombre`	String	Texto no vacío	Se guarda sin espacios al inicio ni al final

El siguiente mensaje es un ejemplo de referencia construido según el contrato y las pruebas unitarias del consumidor. El mensaje real se incorporará cuando el publicador esté desplegado.

```
{
  "eventId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "tipoEvento": "categoria.creada",
  "fechaEvento": "2026-10-29T15:00:00Z",
  "categoriaId": 7,
  "nombre": "Hardware"
}
```

El campo `categoriaId` relaciona el registro original con su copia. Solicitudes no genera una clave nueva, sino que guarda el mismo valor en la columna `categoria_id` de `categorias_locales`. La copia agrega la fecha de recepción, que no forma parte del mensaje.

> *[Pendiente: mensaje real capturado en RabbitMQ o en los logs, con su eventId y categoriaId — responsable Nicolás]*

8.4 Confirmaciones de publicador y consumidor

El flujo utiliza dos confirmaciones distintas. La confirmación del publicador indica que RabbitMQ aceptó el mensaje. Si el broker no encuentra una cola para la routing key, devuelve el mensaje como retornado. Ninguna de estas señales acredita que Solicitudes haya escrito la copia en su RDS.

La confirmación del consumidor ocurre en Solicitudes. El listener trabaja con acuse manual y envía `basicAck` solo después de que la copia queda guardada en RDS Solicitudes. Mientras no exista ese acuse, RabbitMQ mantiene el mensaje como Unacked y lo vuelve a entregar si el consumidor se desconecta.

Tabla 8.4. *Comparación entre ambas confirmaciones.*

Aspecto	Confirmación del publicador	Confirmación del consumidor
Quién la emite	RabbitMQ hacia Catálogo	Solicitudes hacia RabbitMQ
Qué acredita	Aceptación del mensaje por el broker	Persistencia de la copia en RDS Solicitudes
Mecanismo	Publisher confirms y retorno de mensajes sin ruta	basicAck manual después de guardar
Estado	Pendiente de implementación en ms-catalogo	Implementado en ms-solicitudes

Para que un mensaje sobreviva al reinicio del broker deben cumplirse tres condiciones. La cola debe ser durable, el mensaje debe publicarse como persistente y el volumen de `/var/lib/rabbitmq` debe conservarse. La cola y el volumen ya cumplen esa condición, y la publicación persistente depende del publicador.

8.5 Manejo de fallas

La Tabla 8.5 resume el comportamiento previsto ante las fallas más relevantes del flujo.

Tabla 8.5. *Comportamiento ante fallas.*

Situación	Comportamiento	Efecto
Evento con campos faltantes o tipo distinto	basicNack sin reencolar	El mensaje se descarta y queda registrado en el log
Error al guardar en RDS Solicitudes	basicNack con reencolado	El mensaje vuelve a la cola para un nuevo intento
Consumidor detenido	El mensaje queda Ready en la cola	Se procesa cuando el consumidor vuelve a iniciar
Entrega repetida del mismo categoriaId	La copia se actualiza sobre la misma fila	No se duplican categorías locales
Broker detenido al crear una categoría	Respuesta controlada, pendiente de implementar en el publicador	La categoría puede quedar en RDS Catálogo sin evento

El guardado en RDS Catálogo y la publicación son operaciones separadas. Si la primera tiene éxito y la segunda falla, la categoría existe en Catálogo pero no llega a Solicitudes. El publicador deberá responder con un error controlado e indicar que el registro pudo quedar guardado, sin afirmar una reversión automática. Si el administrador reenvía la solicitud, puede crear una segunda categoría con otro `categoriaId`.

En el consumidor, un error persistente de base de datos provoca reintentos sucesivos, ya que el proyecto no incluye una cola de mensajes fallidos. Una entrega repetida actualiza la misma fila porque la escritura usa `categoriaId` como clave. Esto evita duplicados, pero el sistema no garantiza un procesamiento exactamente una vez.

9. Implementación del consumidor en Solicitudes

El consumidor está implementado en ms-solicitudes con Spring AMQP, a partir del patrón de consumidor del laboratorio adaptado al dominio MesaTech. El código se encuentra en una rama de trabajo del repositorio y su integración a la rama de desarrollo está pendiente.

Tabla 9.1. *Componentes del consumidor.*

Componente	Responsabilidad
`RabbitConfig`	Convierte el JSON con Jackson y declara exchange, cola y binding solo si RABBITMQ_DECLARE es true
`CategoriaCreadaEvent`	Representa los cinco campos del contrato
`CategoriaCreadaListener`	Escucha solicitudes.catalogo.queue, invoca la persistencia y envía el acuse
`CategoriaCopiaService`	Valida el evento y guarda o actualiza la copia dentro de una transacción
`CategoriaLocal`	Entidad de la tabla categorias_locales, con categoria_id, nombre, event_id, fecha_evento y recibida_en
`SolicitudController`	Expone GET /v1/solicitudes/categorias y valida la categoría al crear una solicitud

9.1 Secuencia de procesamiento

Cada mensaje recorre los siguientes pasos dentro de ms-solicitudes.

1. El listener recibe el mensaje y Jackson lo convierte en `CategoriaCreadaEvent`.
2. `CategoriaCopiaService` valida los cinco campos y el tipo de evento.
3. El servicio busca la categoría por `categoriaId` y crea o actualiza la fila correspondiente.
4. Después de guardar, el listener envía `basicAck` y registra un log con `categoriaId`, nombre y `eventId`, sin secretos.
5. Si la validación falla, envía `basicNack` sin reencolar. Si falla la base de datos, envía `basicNack` con reencolado.

El log de un consumo correcto tiene la forma «Categoría <categoriaId> (<nombre>) persistida desde el evento <eventId>». Este registro permite relacionar el evento con la fila persistida sin leer el RDS de Catálogo.

9.2 Uso de la copia local

El endpoint `GET /v1/solicitudes/categorias` devuelve las categorías locales ordenadas por nombre y queda protegido por el mismo flujo JWT de la EP1, a través del BFF. Al registrar una solicitud, Solicitudes verifica que la categoría exista en la copia local. Si todavía no llegó, la API responde 409 Conflict.

En React, el formulario de creación lista solo las categorías recibidas y ofrece actualizar el listado. Así, el usuario reconoce cuándo una categoría recién creada ya está disponible para registrar una solicitud.

> *[Pendiente: ruta GET /v1/solicitudes/categorias en API Gateway con el mismo autorizador de /v1/solicitudes — responsable Giannina]*

9.3 Verificación en entorno local

El código incluye pruebas unitarias del listener y del servicio de copia. Estas pruebas cubren el acuse después de persistir, el rechazo sin reencolar de eventos inválidos, el reencolado ante errores de persistencia y la conservación de `categoriaId`.

Como el consumidor aún no podía desplegarse en la EC2 B, el equipo ejecutó una copia local de ms-solicitudes con la misma configuración de usuarios de Solicitudes y publicó un evento de prueba. La Tabla 9.2 registra el resultado.

Tabla 9.2. *Prueba funcional del consumidor en entorno local.*

Dato	Valor
Fecha	1 de noviembre de 2026
Entorno	Local. No reemplaza la evidencia en AWS
Fila obtenida en categorias_locales	categoria_id 42, nombre Redes-local

> *[Pendiente: despliegue del JAR de ms-solicitudes en la EC2 B, conectado a RDS Solicitudes y al broker — responsable Skarlett]*

9.4 Estado del publicador en Catálogo

El publicador de ms-catalogo aún no está implementado en el repositorio. Su diseño queda definido por el contrato y por la sección 8. Deberá validar la solicitud, guardar la categoría, publicar un mensaje persistente con la routing key `categoria.creada`, activar las confirmaciones y los retornos, y responder con un error controlado si el broker no confirma.

> *[Pendiente: implementación del publicador en ms-catalogo y despliegue del JAR en la EC2 A — responsable Nicolás]*

10. Continuidad con la EP1

La EP2 mantiene la capa de acceso seguro de la EP1. React autentica a la persona con MSAL frente a Microsoft Entra ID y envía el token como Bearer a API Gateway. El Gateway es la única puerta pública de las APIs y reenvía las llamadas al BFF, que valida el JWT y los roles antes de coordinar con los microservicios.

Tabla 10.1. *Elementos de la EP1 que se conservan.*

Elemento	Comportamiento conservado	Verificación
React con MSAL	Inicio de sesión con Entra ID y envío del token en cada llamada	P8
Microsoft Entra ID	App roles Cliente, Operador y Administrador	P8
API Gateway	Única puerta pública, con rutas v1 y v2	P8
BFF	Validación JWT y autorización por rol, sin acceso a RDS	P8
Reglas de estado	RESUELTA solo se alcanza desde EN_PROCESO	P8
Versionamiento	GET /v1/solicitudes/mias y GET /v2/solicitudes/mias disponibles	P8

El BFF no publica ni consume mensajes y no se conecta a ninguna RDS. Con la nueva topología, sus destinos internos pasan a ser las direcciones privadas de la EC2 A y de la EC2 B.

> *[Pendiente: capturas de rechazo 401 sin token y respuesta 200 con token a través de API Gateway en la nueva topología — responsable Giannina]*

11. Plan y resultados de pruebas P1 a P8

Cada prueba registra la preparación, los datos, el resultado esperado, el resultado obtenido y la evidencia. Las pruebas P2 a P5 utilizan categorías nuevas e identificables para seguir cada caso en RabbitMQ y en ambas RDS.

Durante las pruebas de espera y reinicio no se utiliza Get messages con una confirmación que retire el mensaje. Si se inspecciona el contenido desde la interfaz, se usa la opción que lo devuelve a la cola.

Tabla 11.1. *Pruebas P1 a P4.*

Prueba	Preparación y datos	Resultado esperado	Resultado obtenido	Evidencia
P1 Separación real	Inventario de EC2 y RDS; proceso de cada microservicio con su conexión	IDs distintos de EC2 A y B, endpoints RDS distintos y cada proceso con su conexión y tabla	Pendiente de ejecución	Figuras 5.2 y 5.3; falta proceso y consulta
P2 Flujo completo	Administrador autenticado en React; categoría nueva Redes-EP2	Fila en RDS Catálogo, mensaje publicado, log del consumidor con el eventId, fila local con el mismo categoriaId y solicitud creada con esa categoría	Pendiente de ejecución	Capturas de React, ambas RDS, cola y log
P3 Consumidor detenido	ms-solicitudes detenido en la EC2 B; otra categoría nueva	Categoría en RDS Catálogo, mensaje Ready en la cola y sin fila en la copia local	Pendiente de ejecución	Captura de la cola y consulta local
P4 Consumo diferido	Inicio del consumidor con el mensaje de P3 pendiente	Fila guardada en RDS Solicitudes y cola sin mensajes Ready ni Unacked para ese caso	Pendiente de ejecución	Log del acuse, fila local y cola

Tabla 11.2. *Pruebas P5 a P8.*

Prueba	Preparación y datos	Resultado esperado	Resultado obtenido	Evidencia
P5 Reinicio del broker	Consumidor detenido y mensaje pendiente; contenedor recreado con el mismo volumen y hostname	Se conservan virtual host, exchange, cola y mensaje; al iniciar el consumidor aparece la fila	Pendiente de ejecución	Cola antes y después, compose ps y fila local
P6 Reinicio de aplicaciones	Filas existentes en ambas RDS; reinicio de ms-catalogo y ms-solicitudes	Los mismos registros en ambas RDS después del reinicio	Pendiente de ejecución	Consultas antes y después en ambas RDS
P7 Fallo de publicación	RabbitMQ detenido; intento de crear una categoría	Respuesta controlada del publicador, consulta a RDS Catálogo con el resultado real y ausencia de copia en Solicitudes	Pendiente de ejecución	Respuesta HTTP, RDS Catálogo y copia local
P8 Continuidad EP1	Usuarios con distintos roles y una llamada sin token	Login y consumo autorizado, rechazo sin token y por rol, bloqueo de RESUELTA sin EN_PROCESO, v1 y v2 disponibles y categoría recibida utilizable	Pendiente de ejecución	Capturas 401, 403, transición y v1/v2

Para P5, el contenedor se recrea con `docker compose up -d --force-recreate`, sin usar la opción que elimina volúmenes, de modo que el volumen `rabbitmq_data` y el hostname se mantienen.

> *[Pendiente: ejecución y capturas de P1 a P8 según la bitácora de pruebas, con aportes de Ari, Nicolás y Giannina — responsable Skarlett]*

11.1 Consulta de verificación de la copia local

La siguiente consulta se ejecuta desde la EC2 B, porque RDS Solicitudes no tiene acceso público. Permite verificar en P2 a P6 la fila de cada categoría recibida.

```
SELECT categoria_id, nombre, event_id, fecha_evento, recibida_en
FROM categorias_locales
ORDER BY recibida_en;
```

11.2 Reflexión sobre publicación, confirmaciones, persistencia y fallos

La principal conclusión del equipo es que una publicación confirmada no equivale a un dato replicado. La confirmación del publicador solo indica que el broker aceptó el mensaje. La replicación termina cuando el consumidor guarda la copia en RDS Solicitudes y envía su acuse, por lo que el acuse manual después de persistir es el punto que protege la consistencia de la copia local.

La persistencia del broker depende de tres decisiones que deben coincidir. Una cola durable con mensajes no persistentes pierde su contenido al reiniciar, y los mensajes persistentes también se pierden si el contenedor se recrea sin conservar el volumen. El hostname estable completa esa condición, porque RabbitMQ asocia su directorio de datos al nombre del nodo.

El punto débil del flujo está entre el guardado en Catálogo y la publicación. Sin outbox ni transacciones distribuidas, una caída del broker en ese intervalo deja una categoría sin copia. El equipo asume ese riesgo de forma explícita, lo documenta en P7 y lo hace visible con una respuesta controlada.

En el consumidor, la escritura por `categoriaId` evita que una entrega repetida duplique categorías, aunque no constituye una garantía de procesamiento exactamente una vez. Estas conclusiones se basan en el diseño y en el código del consumidor, y el equipo las contrastará con los resultados de P3 a P7 cuando el flujo completo esté desplegado en AWS.

12. Limitaciones y trabajo pendiente

Al cierre de este informe, la infraestructura y el broker están configurados, y el consumidor está implementado y verificado en entorno local. El flujo completo en AWS depende de los pendientes de la Tabla 12.1.

Tabla 12.1. *Trabajo pendiente, responsable e impacto.*

Pendiente	Responsable	Impacto
Acceso de despliegue a la EC2 B y regla del BFF hacia el puerto 8081	Ari	Bloquea el despliegue de Solicitudes y las pruebas P1, P2, P4 y P6
Publicador de categoria.creada en ms-catalogo con confirmaciones y retornos	Nicolás	Bloquea P2 a P5 y P7
Despliegue del JAR de Catálogo en la EC2 A y ubicación del BFF	Nicolás	Bloquea P1, P2, P6 y P7
Despliegue del JAR de Solicitudes en la EC2 B	Skarlett	Bloquea P1 a P6
Ruta GET /v1/solicitudes/categorias en API Gateway con el mismo autorizador de /v1/solicitudes	Giannina	Bloquea el uso de la copia local desde React en P2 y P8
Capturas 401 y 200 a través de API Gateway	Giannina	Completa la evidencia de P8
compose.yaml en el repositorio, permisos de usuarios, logs, diagnóstico y firewall del sistema operativo	Ari	Completa la evidencia de instalación, red y mensajería
Usuarios de aplicación en cada RDS	Ari	Completa la configuración de las bases
Integración del consumidor a la rama de desarrollo	Skarlett	Completa el código entregable
Ejecución y capturas de P1 a P8	Skarlett, con el equipo	Completa los resultados de la sección 11

El proyecto no implementa outbox, cola de mensajes fallidos, idempotencia por `eventId` ni recuperación automática ante fallas de publicación, ya que están fuera del alcance de la evaluación. La prueba de reinicio del broker evalúa la conservación local de los datos y no alta disponibilidad ni recuperación ante la pérdida de la EC2.

Las IP públicas del laboratorio cambian cada vez que se reinician las instancias. Por esa razón, algunas capturas tempranas de la interfaz de RabbitMQ se tomaron con una dirección distinta de la vigente, y el informe omite todas las direcciones IP.

13. Conclusiones

La EP2 modificó la forma en que Catálogo y Solicitudes se integran. En lugar de consultar la API de Catálogo, Solicitudes mantiene una copia local alimentada por eventos, con infraestructura separada en tres EC2 y dos instancias RDS exclusivas. Esta separación respeta el principio de que cada servicio es dueño de sus datos.

La instalación de RabbitMQ desde cero con Docker Compose, los recursos del virtual host mesatech y las reglas de los security groups quedaron configurados y acreditados con capturas. El consumidor confirma cada mensaje solo después de guardar la copia y conserva el `categoriaId` de origen, lo que permite rastrear cada categoría entre ambas bases.

El trabajo pendiente se concentra en el publicador, el despliegue de los JAR y la ejecución de P1 a P8. El equipo dejó identificados los responsables y las evidencias que completarán el informe, y mantiene la seguridad de la EP1 como condición para validar el flujo desde React.

14. Responsabilidades del equipo

La Tabla 14.1 detalla la distribución de tareas acordada para la EP2 y los aportes que respaldan este informe.

Tabla 14.1. *Responsabilidades de cada integrante.*

Integrante	Responsabilidad en la EP2	Aportes
Ari Araya	Infraestructura en AWS Academy	Tres EC2, dos RDS, Docker y Compose en la EC2 C, compose.yaml, recursos de RabbitMQ, security groups e inventario
Nicolás Barra	Publicador y despliegue de Catálogo	Publicador en ms-catalogo, despliegue en la EC2 A y ubicación del BFF, en curso
Giannina Guerrero	Continuidad de la EP1, diagrama y documentación	Diagrama de despliegue, contrato del evento y continuidad de API Gateway, Entra ID y BFF
Skarlett Tropan	Consumidor, pruebas e informe	Consumidor en ms-solicitudes, copia local, prueba en entorno local, bitácora P1 a P8 y borrador del informe

Referencias

Amazon Web Services. (s. f.). Controlling access with security groups. Amazon RDS Getting Started Guide. https://docs.aws.amazon.com/AmazonRDS/latest/gettingstartedguide/security-groups.html

Docker Inc. (s. f.). Install Docker Engine on Ubuntu. https://docs.docker.com/engine/install/ubuntu/

Docker Inc. (s. f.). Define services in Docker Compose. https://docs.docker.com/reference/compose-file/services/

RabbitMQ. (s. f.). Consumer acknowledgements and publisher confirms. https://www.rabbitmq.com/docs/confirms

RabbitMQ. (s. f.). Networking and RabbitMQ. https://www.rabbitmq.com/docs/networking

RabbitMQ. (s. f.). Persistence configuration. https://www.rabbitmq.com/docs/persistence-conf

Duoc UC. (2026). Material de la asignatura DSY1107 Desarrollo Cloud Native I. Guías de instalación de un servidor RabbitMQ y de publicador y consumidor.